



Universidad de Cuenca

Facultad de Ingeniería
Ingeniería en Telecomunicaciones

Microprocesadores, Microcontroladores y Sistemas Embebidos

Kenneth S. Palacio Baus
kenneth.palacio@ucuenca.edu.ec

Jaime Patricio Chiqui Chiqui
jpatricio.chiquic@ucuenca.edu.ec

Práctica 1

PIC18F4550 - Manejo de Entradas y Salidas Digitales

- **Valoración:** 10 puntos.
- **Tipo de Trabajo:** Individual.
- **Objetivos:**
 - Reconocer los elementos de hardware mínimos para el funcionamiento de un sistema microcontrolado.
 - Comprender el funcionamiento y estilo de programación del lenguaje ensamblador.
 - Realizar operaciones básicas de entrada y salida digitales con el microcontrolador.
- **Recursos:** Como base de esta práctica, utilizaremos MPLAB y la hoja de datos del microcontrolador PIC18F4550.

Instrucciones

Para obtener una calificación en la presente práctica, cada estudiante deberá entregar un informe impreso según la estructura que se menciona más adelante. No olvide que puede contactar al profesor vía correo electrónico en caso que necesite asistencia adicional.

- Envíe su trabajo mediante la plataforma e-nova.
- El nombre del archivo de su informe debe tener el formato:
ApellidoNombre_Pract01_MicroCon_M26.pdf
- Si su envío no cumple con el nombre de archivo y fecha de entrega, no recibirá calificación.
- Si su informe no se encuentra en las dos plataformas indicadas, no recibirá calificación.

1. Materiales y Equipos

Para el desarrollo de la presente práctica, se utilizaron los siguientes recursos y componentes electrónicos:

- **Software:** MPLAB IDE v8.92.
- **Microcontrolador:** PIC18F4550 de Microchip.
- **Resistencias:**
 - 3 de 10 k Ω (Pull-up para pulsantes y reset).
 - 9 de 1 k Ω (Limitadoras para los LEDs).
- **Oscilador:** Cristal de cuarzo de 20 MHz (XTAL).
- **Capacitores:** 2 cerámicos de 22 pF (para el cristal).
- **Pulsantes:** 3 micropulsantes (S1, S2, S3).
- **Visualización:** 9 diodos LED.
- **Programador** *PICkit 3* con su respectivo cable USB.
- **Espadín:** Conector de 6 pines largos pines tipo espadín (peineta).
- **Otros:** Protoboard, cables de conexión calibre 22 (o jumpers).

2. Marco Teórico

2.1. Arquitectura y Gestión de E/S del PIC18F4550

El PIC18F4550 es un microcontrolador robusto que permite el manejo de señales digitales mediante sus puertos. Para la manipulación de entradas y salidas, es fundamental comprender la interacción entre los registros de configuración. En la Figura 1 se presenta el diagrama de pines del dispositivo, identificando los pines RC0 y el Puerto D utilizados en esta práctica.

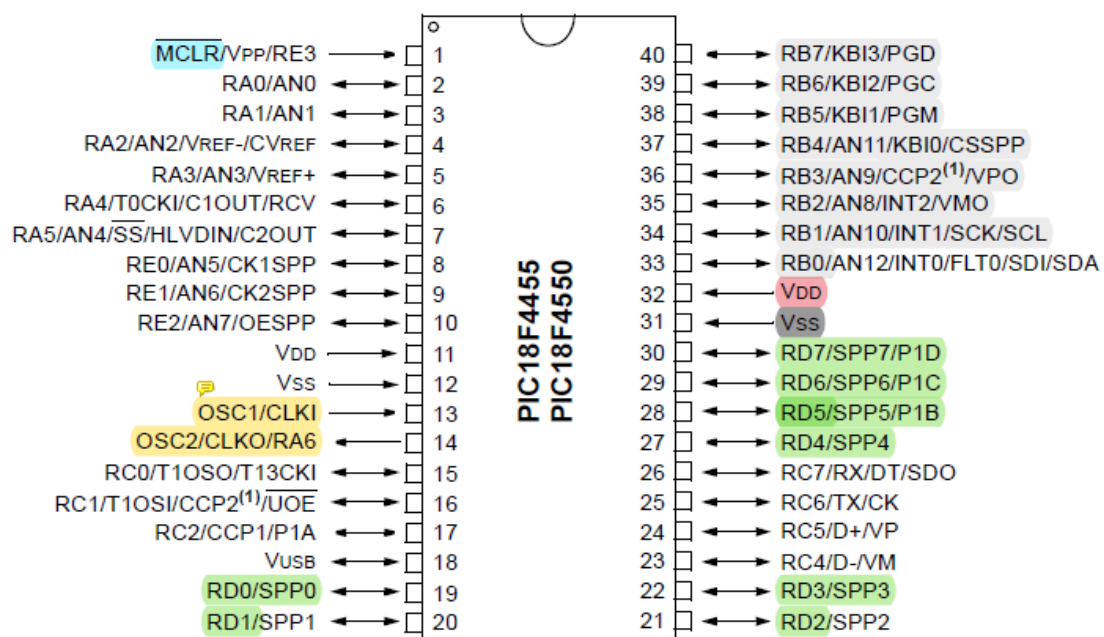


Figura 1: Diagrama de pines y distribución del PIC18F4550, obtenido del datasheet.

Para la implementación de este diseño, tomé en consideración las siguientes especificaciones críticas del microcontrolador:

- **Procesamiento:** Emplea una arquitectura tipo RISC de alta eficiencia, optimizada para ejecutar un repertorio de 75 instrucciones.
- **Capacidades de Memoria:** Dispone de espacio suficiente para nuestras secuencias gracias a sus 32 KB de memoria Flash, acompañados de 2 KB de RAM y 256 bytes de memoria EEPROM.
- **Gestión de Reloj:** Ofrece flexibilidad en la temporización; puede funcionar con su propio oscilador interno (8 MHz) o sincronizarse mediante un cristal externo de hasta 20 MHz, como el empleado en el circuito físico.
- **Condiciones Eléctricas:** Su rango de voltaje de alimentación es de 2.0 V a 5.5 V. Además, cada pin de entrada/salida puede manejar una corriente máxima segura de 25 mA, dato vital para el cálculo de las resistencias limitadoras de los LEDs.
- **Arquitectura tipo Harvard:** Al analizar la estructura interna del dispositivo, es fundamental destacar que no utiliza la arquitectura tradicional (Von Neumann), sino que implementa una arquitectura Harvard. Esta configuración utiliza buses físicos y espacios de almacenamiento independientes para la memoria de programa (instrucciones) y la memoria de datos. En la práctica, esta separación resulta crucial, ya que le permite al microcontrolador leer una instrucción y procesar un dato de forma simultánea, optimizando significativamente la velocidad de ejecución de nuestras rutinas en lenguaje ensamblador. En la Figura 2 se ilustra este principio de funcionamiento.

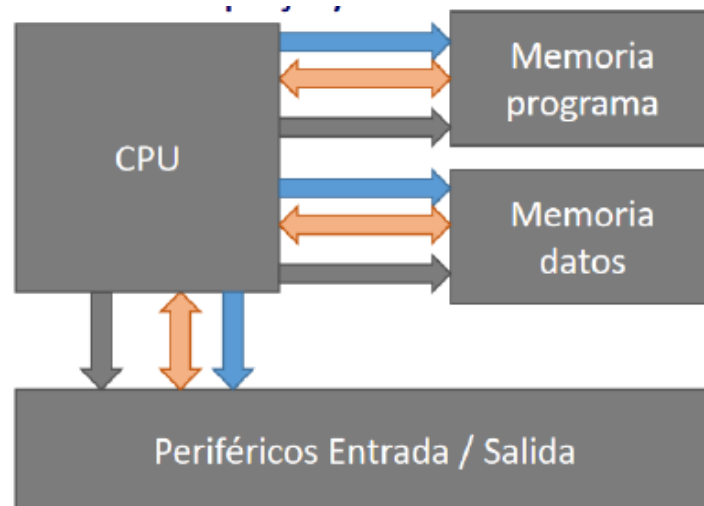


Figura 2: Diagrama de bloques de la arquitectura tipo Harvard implementada en la familia PIC18.

Gestión de Puertos Digitales

En el lenguaje ensamblador, la configuración se basa en:

- **TRIS:** Determina si el pin es Entrada (1) o Salida (0).
- **LAT:** Registro de *Latch* para escritura lógica, evitando errores de lectura/escritura.
- **PORT:** Utilizado primordialmente para la lectura de estados externos.

2.2. Interfaz de Programación: PICKit 3

Para transferir el código desde la computadora al microcontrolador, se utiliza el programador *PICKit 3*. Este dispositivo emplea el protocolo ICSP, conectándose a través de una hilera de 6 pines (espada). Una característica crítica es la marca de orientación (flecha blanca), la cual indica el Pin 1.

En la Figura 3 se detalla la función de cada pin del conector ICSP del programador.

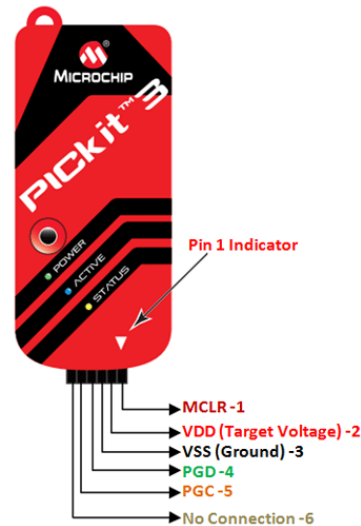


Figura 3: Programador PICKit 3 y detalle de su conector de 6 pines.

2.3. Entorno de Desarrollo: MPLAB IDE v8.92

El desarrollo del *firmware* se realizó en el entorno integrado de desarrollo MPLAB IDE en su versión 8.92. Esta versión es estándar para el aprendizaje de lenguaje ensamblador (MPASM), permite un control absoluto sobre el mapa de memoria y los ciclos de instrucción de la CPU.

Configuración de Bits: Es indispensable configurar los *Configuration Bits* en MPLAB para indicarle al PIC que trabajará con un cristal externo y desactivar funciones como el *Watchdog Timer* (WDT), garantizando que el programa no se reinicie inesperadamente y que estas últimas configuraciones estén como **is not write code-protect**, caso contrario no se podrá usar el pic18f4550.

2.4. Sincronización y Configuración del Oscilador

La velocidad de ejecución depende del sistema de reloj. Para esta práctica, se configuró un oscilador de alta velocidad (HS) utilizando un cristal de cuarzo externo. La frecuencia del oscilador (F_{osc}) se divide internamente para obtener la frecuencia de instrucción (F_{cy}), que es la velocidad real a la que se ejecutan las líneas de código.

La relación matemática es la siguiente:

$$F_{cy} = \frac{F_{osc}}{4} \quad (1)$$

Para un cristal de 20 MHz, los cálculos de tiempo son:

$$T_{cy} = \frac{1}{F_{cy}} = \frac{4}{20 \text{ MHz}} = 0,2\mu s$$

¿Cómo saber qué capacitor usar?

Para asegurar la estabilidad del reloj, los capacitores asociados al cristal de cuarzo no se eligen al azar, sino que deben seleccionarse en función de las especificaciones del fabricante. Al consultar la hoja de datos del PIC18F4550, encontramos una tabla específica para la selección de capacitores en cristales de cuarzo. En nuestro caso, al utilizar un cristal de **20 MHz** (configurado en modo **HS** o *High Speed*), la tabla sugiere utilizar capacitores cerámicos de un valor de **22 pF**, lo cual garantiza que el lazo de realimentación del oscilador interno mantenga una señal estable y precisa, vital para que las rutinas de demora (delays) en ensamblador duren exactamente el tiempo calculado.

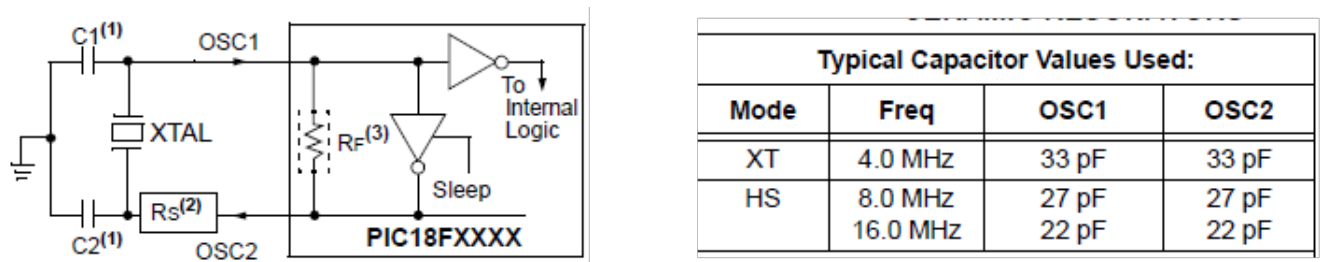


Figura 4: Tabla de selección de capacitores para el oscilador según la frecuencia y su esquema, obtenida de la hoja de datos del fabricante.

3. Procedimiento

1. Implemente el circuito de la Figura 5 en un project board.

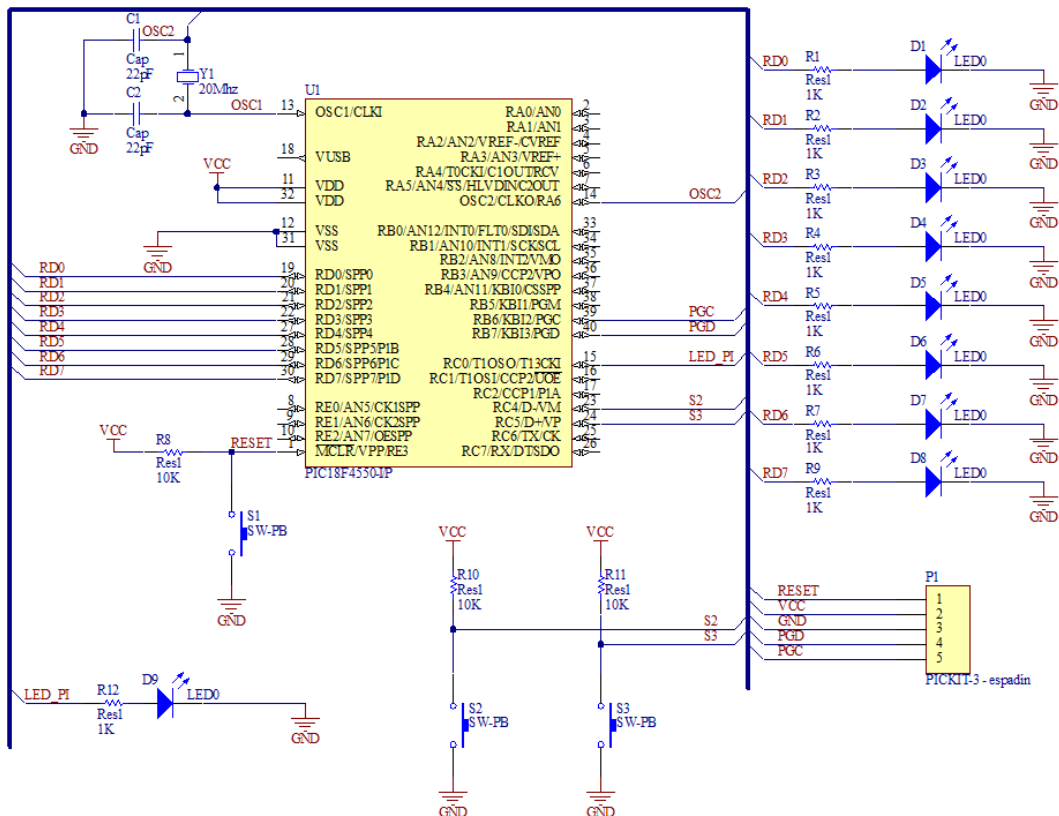


Figura 5: Circuito para manejo de E/S PIC18F4550

En la Figura 5 se puede apreciar el diagrama implementado en una protoboard.

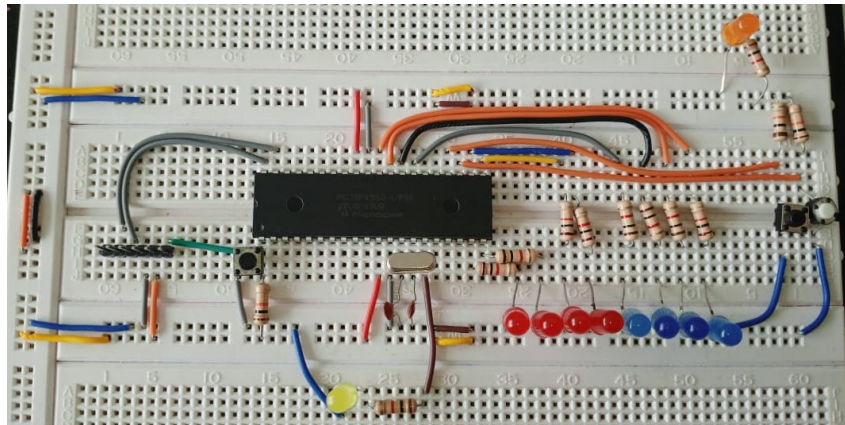


Figura 6: Circuito implementado en protoboard.

Descripción de componentes utilizados:

- **PIC18F4550:** Ejecuta la lógica de las secuencias mediante instrucciones en ensamblador.
- **Resistencias:** Limita la corriente hacia los LEDs (evita que se quemen) y para los pulsantes, asegurando un nivel lógico alto constante cuando no están presionados.
- **Pulsantes:** Actúan como las interfaces de entrada digital que permiten al usuario interactuar con el programa para cambiar o detener las secuencias de luces.
- **LEDs:** Actúan como periféricos de salida para la visualización de los datos procesados.
- **Cristal de Cuarzo y Capacitores:** Forman el circuito oscilador externo que proporciona la señal de reloj necesaria para que el microcontrolador sincronice sus operaciones internas.

2. *Utilice MPLAB IDE v8.92 para desarrollar un programa en lenguaje ensamblador que cumpla con las siguientes condiciones que se le indiquen a continuación.*
3. *Utilizará subrutinas de demora implementadas previamente por software, por ejemplo de 1ms, etc.*

```
1 ;++++++ DELAY DE 1MS ++++++
2 Delay1ms:
3     DECFSZ    Contador2,1
4     GOTO      bucle1
5     GOTO      salir
6 bucle1:
7     DECFSZ    Contador1,1
8     GOTO      bucle1
9
10    MOVLW     d'198'      ;W = 198
11    MOVWF     Contador1   ;Contador1 = 198
12    GOTO      Delay1ms
13 salir:
14    MOVLW     d'21'       ;W = 21
15    MOVWF     Contador2   ;Contador2 = 21
16    return
```

Listing 1: Subrutina de retardo de 1ms

Explicación: Esta subrutina implementa un retardo de aproximadamente 1 ms utilizando dos bucles anidados. El contador interno (Contador1) se decrementa 198 veces, y el

contador externo (Contador2) se decrementa 21 veces. Con un cristal de 20 MHz ($T_{cy} = 0.2 \mu s$), el cálculo aproximado es: $198 \times 21 \times 0,2 \mu s \approx 1 ms$. Esta es la base para todas las demoras del programa.

Delay de X milisegundos:

```

1 ;++++++ DELAY DE X[ms] ++++++
2 DelayXms:
3     MOVWF    Xveces
4 bucleVeces:
5     CALL     Delay1ms
6     DECFSZ   Xveces,1
7     GOTO     bucleVeces
8     return

```

Listing 2: Subrutina de retardo variable en ms

Explicación: Esta subrutina permite generar retardos de duración variable. Recibe en el registro W la cantidad de milisegundos deseados, los almacena en la variable Xveces, y llama a Delay1ms repetidamente hasta completar el tiempo solicitado. Por ejemplo, para un retardo de 50 ms se carga W con 50 antes de llamar a DelayXms.

Delay de 100ms:

```

1 ;++++++ DELAY DE 100MS ++++++
2 Delay100ms:
3     CALL     Delay1ms
4     DECFSZ   ContaDelay100,1
5     GOTO     Delay100ms
6 ;Reinicio ContaDelay100 = 100
7     MOVLW    d'100' ;W=100
8     MOVWF    ContaDelay100
9     return

```

Listing 3: Subrutina de retardo de 100ms

Explicación: Esta subrutina genera un retardo fijo de 100 ms llamando a Delay1ms un total de 100 veces. Se utiliza principalmente para el antirrebote de los pulsantes, permitiendo que el microcontrolador espere a que se estabilice la señal mecánica del pulsante.

4. *Configure el puerto digital RC0 como salida digital.*

La configuración de los puertos se realiza al inicio del programa mediante los registros TRIS correspondientes:

```

1 ; Configuracion de perifericos
2 CLRF    TRISD      ; TRISD = 00000000 (Puerto D como salida)
3 BCF     UCON, 3     ; RC4 Y RC5 entradas (S2 Y S3)
4 BSF     UCFG, 3     ; Especial RC4 Y RC5
5 BCF     TRISC, 0    ; led_piloto como salida

```

Listing 4: Configuración de periféricos

Explicación: Se configura el Puerto D completo como salida digital (para los 8 LEDs) mediante la instrucción CLRF TRISD, que coloca todos los bits en 0. Los pines RC4 y RC5 se configuran como entradas digitales para los pulsantes S2 y S3 respectivamente, utilizando la configuración especial del módulo USB del PIC18F4550. El pin RC0 se configura como salida para el LED piloto.

5. *Secuencia de inicio: Implemente una secuencia de inicio, en la que un LED conectado al pin digital RC0, se encienda y apague 20 veces, con intervalos de 100ms, es decir, 50ms encendido y 50ms apagado.*


```

1 ;----- SECUENCIA DE INICIO -----
2 MOVLW    d'20'
3 MOVWF    ContadorInicio ;RC0 parpadea 20 veces / 50 ms
4
5 BucleInicio:
6     BSF    LATC, 0          ; Enciende LED RC0-Led_piloto
7     MOVLW    d'50'
8     CALL    DelayXms
9     BCF    LATC, 0          ; Apaga LED RC0-Led_piloto
10    MOVLW    d'50'
11    CALL    DelayXms
12    DECFSZ   ContadorInicio, 1 ; Decrementa y salta si es cero
13    GOTO     BucleInicio
14
15    BSF    LATC, 0          ; RC0-Led_piloto ENCENDIDO

```

Listing 5: Secuencia de inicio - LED piloto parpadeante

Explicación: Al iniciar el programa, el LED conectado a RC0 parpadea 20 veces con intervalos de 100 ms (50 ms encendido, 50 ms apagado). Se utiliza un contador que se decrementa en cada ciclo hasta llegar a cero. Al finalizar, el LED queda encendido de forma permanente, indicando que el sistema está listo para operar.

6. *Al finalizar la secuencia de encendido, el LED en RC0 deberá permanecer encendido.*

Para mantenerlo encendido es mediante **BSF LATC, 0**

PARTE 1: MANEJO DE ENTRADAS Y SALIDAS

Luego de la secuencia de inicio, en esta parte su programa deberá hacer lo siguiente:

- a) *El Puerto D del microcontrolador se debe configurar como salida digital.*

```

1 ; Configuración de perifericos
2 CLRF     TRISD          ; TRISD = 00000000 (Puerto D como salida)

```

Listing 6: Configuración Puerto D como salida

Explicación: La instrucción CLRF TRISD limpia todos los bits del registro TRISD, colocándolos en 0. Esto configura los 8 pines del Puerto D (RD0-RD7) como salidas digitales, permitiendo controlar los 8 LEDs conectados.

- b) *Los pulsantes S2 y S3 sirven como señales manuales de inicio y fin de la secuencia de luces en los LED conectados al puerto D.*

```

1 EncuestaON:          ; Espera a S2 para iniciar secuencia
2     BTFSC    PORTC, 4
3     GOTO     EncuestaON

```

Listing 7: Espera del pulsante S2 para iniciar

Explicación: Este bucle verifica continuamente el estado del pin RC4 (S2). La instrucción BTFSC salta si el bit está en 1 (botón no presionado). Cuando S2 se presiona (RC4 = 0), el programa continúa ejecutando la secuencia.

- c) *Al presionar S2, una secuencia SQ1 de luces debe iniciar.*

Antes de iniciar la secuencia, se inicializan dos variables que controlan el patrón de LEDs. NH representa los 4 bits superiores y NL los 4 bits inferiores del Puerto D. La inicialización es:

- NH = 10000000 (bit 7 encendido)
- NL = 00000001 (bit 0 encendido)

Al combinar ambos con la operación OR, se obtiene el patrón inicial: 10000001 (LEDs externos encendidos).

```

1 Reset_NH_NL:
2     MOVLW    b'10000000'
3     MOVWF    NH
4     MOVLW    b'00000001'
5     MOVWF    NL
6     CLRF     LATD
7
8 Cortina:
9     BTFSS    PORTC, 5           ; S3 es RC5
10    GOTO     Apagar_Todo        ; Si se presiona S3
11    MOVF     NH, W
12    IORWF    NL, W
13    MOVWF    LATD
14
15    MOVLW    d'50'
16    CALL     DelayXms
17
18    RRCF     NH, 1
19    RLNCF    NL, 1

```

Listing 8: Inicialización y ejecución de secuencia SQ1

Explicación: Se inicializan NH y NL con los patrones base. Luego se combinan con OR (IORWF) y se envían al Puerto D. Las rotaciones RRNCF y RLNCF mueven los bits para crear el efecto cortina, sin acarreo.

d) *Al presionar S3, la secuencia de luces debe apagarse (ningún LED encendido).*

```

1 Apagar_Todo:
2     CLRF     LATD                ; Apaga todos los LEDs del puerto RD
3
4 Boton_S3:
5     BTFSS    PORTC, 5
6     GOTO     Boton_S3
7     CALL     Delay100ms
8     BTFSC    PORTC, 5
9     GOTO     Reset_NH_NL

```

Listing 9: Detección de S3 y apagado de LEDs

Explicación: Al detectar S3 presionado (RC5 = 0), se limpia LATD apagando todos los LEDs. Luego espera a que se suelte el botón y reinicia las variables NH y NL para comenzar nuevamente.

e) *Tanto S2 como S3 pueden presionarse en cualquier instante.*

Durante toda la ejecución de la secuencia, el programa verifica continuamente el estado de los pulsantes:

- **S3 (RC5):** Si se presiona, apaga todos los LEDs (CLRF LATD) y espera a que se suelte el botón para reiniciar las variables NH y NL.
- **S2 (RC4):** Permanece monitoreado en la etiqueta EncuestaON, esperando que se presione para iniciar la secuencia.

f) *La secuencia de luces en el Puerto D, denominada SQ1, se muestra a continuación:*

```

LATD = b'10000001'
Delay_50ms
LATD = b'01000010'
Delay_50ms
LATD = b'00100100'
Delay_50ms
LATD = b'00011000'
Delay_50ms
LATD = b'00100100'
Delay_50ms
LATD = b'01000010'
Delay_50ms
LATD = b'10000001'
Delay_50ms
LATD = b'01000010'
Delay_50ms
. . . .
.
.
.... se repite

```

La secuencia implementa un efecto de “cortina” donde los LEDs se encienden desde los extremos hacia el centro y luego regresan. Se utilizan rotaciones circulares:

- RRNCF NH, 1: Rota NH a la derecha (mueve el bit encendido hacia el centro)
- RLNCF NL, 1: Rota NL a la izquierda (mueve el bit encendido hacia el centro)

1. 10000001 → 01000010 → 00100100 → 00011000

2. Al detectar que llegó al centro (NH bit 3 = 1), invierte el movimiento

3. 00011000 → 00100100 → 01000010 → 10000001

PARTE 2: MULTIPLES FUNCIONES

Luego de la secuencia de inicio, en esta parte su programa deberá hacer lo siguiente:

- a) *Incorpore dos secuencias adicionales, llamadas SQ2 y SQ3, personalizadas por cada estudiante.*

Se incorporaron dos secuencias adicionales (SQ2: Ola y SQ3: Luces Policía), permitiendo alternar entre ellas mediante el pulsante S2.

- b) *Estas secuencia se obtienen al presionar S2. Si este botón se presiona cíclicamente, lo que se obtiene es que se ejecutan las secuencias SQ1, SQ2, SQ3, SQ1 ... etc.*

- c) *El funcionamiento de S3 permanece inalterado, es decir, que detiene cualquier secuencia activa, apagando todos los LED, quedando el programa a la espera de que se presione nuevamente S2.*

- d) *Note que al presionar S3, el programa siempre debe regresar a condiciones iniciales.*

El cambio principal en el código de a parte 1, es la implementación de un sistema de estados.

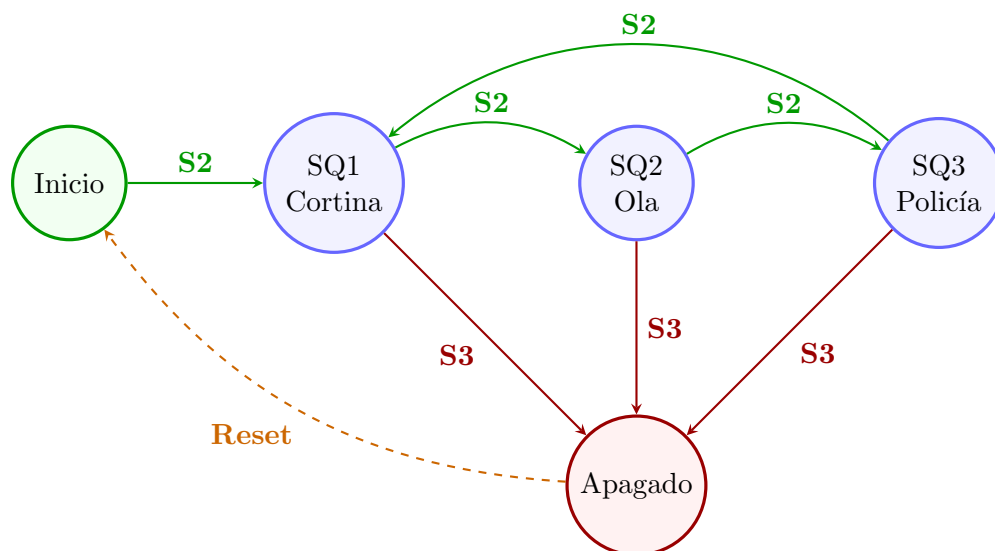


Figura 7: Diagrama de máquina de estados para las tres secuencias

Nueva variable de estado:

```
1 B_Estados    equ 0x07    ; Para los 3 estados
```

Esta variable almacena el estado actual (1=Cortina, 2=Ola, 3=Policía) y se inicializa en 1.

Sistema de selección de secuencia:

```

1 Opcion_sec:
2   MOVLW    d'1'
3   SUBWF    B_Estados, W    ; compara estado con 1
4   BTFSC    STATUS, Z
5   GOTO     Cortina
6
7   MOVLW    d'2'
8   SUBWF    B_Estados, W    ; compara estado con 2
9   BTFSC    STATUS, Z

```

```

10      GOTO    Ola
11
12      MOVLW   d'3'
13      SUBWF   B_Estados, W      ; compara estado con 3
14      BTFSC   STATUS, Z
15      GOTO    Luces_Policia

```

Este bloque compara el valor de B_Estados con 1, 2 y 3 sucesivamente, saltando a la secuencia correspondiente cuando encuentra coincidencia.

Incremento cíclico de estados:

```

1 Opc_fuera:
2   BTFSS     PORTC, 4
3   GOTO      Opc_fuera
4
5   INCF      B_Estados, 1      ; Incrementa estado
6   MOVLW     d'4'
7   SUBWF     B_Estados, W      ; Si llega a 4, volver a 1
8   BTFSC     STATUS, Z
9   GOTO      opcion1
10  GOTO      Opcion_sec
11
12 opcion1:
13  MOVLW      d'1'
14  MOVWF      B_Estados
15  GOTO      Opcion_sec

```

Cuando se presiona S2 durante una secuencia, se incrementa B_Estados. Si llega a 4, regresa a 1, creando un ciclo infinito entre las tres secuencias (1, 2, 3, 1,).

Secuencia SQ2: Ola

```

1 Ola:
2   MOVLW     b'00000001'      ; LED izq ON (rojos)
3   MOVWF     LATD
4 Ola_Hacia_Izq:
5   MOVLW     d'50'
6   CALL      DelayXms
7
8   BTFSS     PORTC, 5          ; verificar boton s3
9   GOTO      Apagar_Todo
10  BTFSS     PORTC, 4
11  GOTO      Opc_fuera
12
13  RLNCF      LATD, 1          ; Rota la luz a la izquierda
14  BTFSS     LATD, 7          ; 10000000 (bit 7 en 1)
15  GOTO      Ola_Hacia_Izq
16
17 Ola_Hacia_Der:
18  MOVLW     d'50'
19  CALL      DelayXms
20
21  BTFSS     PORTC, 5          ; verificar boton s3
22  GOTO      Apagar_Todo
23  BTFSS     PORTC, 4
24  GOTO      Opc_fuera
25
26  RRNCF      LATD, 1          ; Rota la luz a la derecha
27  BTFSS     LATD, 0          ; 00000001 (bit 0 es 1)
28  GOTO      Ola_Hacia_Der
29
30  GOTO      Ola              ; Repite la Ola

```

Listing 10: Secuencia Ola - SQ2

Explicación: Esta secuencia crea un efecto de "ola" donde un solo LED se desplaza de izquierda a derecha y viceversa. Inicia con el LED más a la derecha (bit 0) encendido, luego rota el bit hacia la izquierda usando RLNCF hasta llegar al bit 7. Una vez allí, invierte la dirección usando RRNCF para regresar al bit 0, creando un movimiento continuo. [En mi caso se observa que se enciende desde la izquierda a la derecha es debido a que al implementar en la protoboard se colocaron en la orden diferente](#)

Secuencia SQ3: Luces Policía

```

1  Luces_Policia:
2      ; 3 Parpadeos Rojos (Izquierda)
3      MOVLW    d'3'
4      MOVWF    ContaCiclos
5  Rojos_Parpadean:
6      MOVLW    b'00001111'
7      MOVWF    LATD
8      MOVLW    d'50'
9      CALL     DelayXms
10
11     BTFSS     PORTC, 5          ; Boton s3
12     GOTO      Apagar_Todo
13     BTFSS     PORTC, 4
14     GOTO      Opc_fuera
15
16     CLRF      LATD              ; Apaga Leds
17     MOVLW    d'50'
18     CALL     DelayXms
19     DECFSZ    ContaCiclos, 1    ; resta al contador
20     GOTO      Rojos_Parpadean
21
22     ; 3 Parpadeos Azules (Derecha)
23     MOVLW    d'3'
24     MOVWF    ContaCiclos
25  Azules_Parpadean:
26     MOVLW    b'11110000'
27     MOVWF    LATD
28     MOVLW    d'50'
29     CALL     DelayXms
30
31     BTFSS     PORTC, 5          ; Boton s3
32     GOTO      Apagar_Todo
33     BTFSS     PORTC, 4
34     GOTO      Opc_fuera
35
36     CLRF      LATD              ; Apaga Leds
37     MOVLW    d'50'
38     CALL     DelayXms
39     DECFSZ    ContaCiclos, 1
40     GOTO      Azules_Parpadean
41
42     GOTO      Luces_Policia

```

Listing 11: Secuencia Luces Policía - SQ3

Explicación: Esta secuencia simula las luces de una patrulla. Primero parpadean 3 veces los 4 LEDs inferiores (bits 0-3, representando luces rojas) con intervalos de 50 ms encendido/apagado. Luego parpadean 3 veces los 4 LEDs superiores (bits 4-7, representando luces azules). El contador ContaCiclos controla que sean exactamente 3 parpadeos en cada color antes de alternar.

3.1. PARTE 3: Implementación con Máquina de Estados

La Parte 3 representa una mejora arquitectónica del código de la Parte 2, implementando una verdadera máquina de estados mediante el uso de subrutinas y llamadas con CALL/RETURN.

3.1.1. Diferencias Arquitectónicas con la Parte 2

Parte 2: Utiliza saltos directos (GOTO) entre secuencias, lo que dificulta el control del flujo y requiere verificar botones dentro de cada secuencia.

Parte 3: Implementa cada secuencia como una subrutina que retorna al punto de decisión, permitiendo un control centralizado.

3.1.2. Diagrama de Máquina de Estados Mejorada

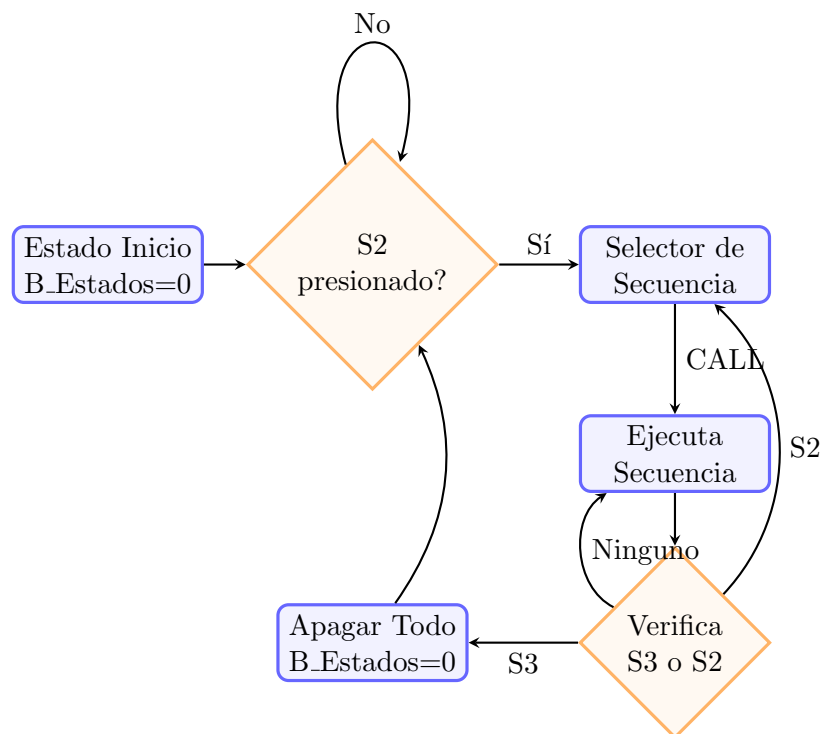


Figura 8: Diagrama de flujo de la máquina de estados - Parte 3

3.1.3. Cambios en el Código anterior

1. Inicialización del estado en 0:

```
1 MOVLW    d'0'           ; Inicia con estado 0
2 MOVWF    B_Estados
```

Se inicia en estado 0, lo que permite un mejor control del inicio.

2. Uso de CALL en lugar de GOTO:

```
1 Opcion_sec:
2     MOVLW    d'1'
3     SUBWF    B_Estados, W
4     BTFSC    STATUS, Z
5     CALL     Cortina           ; Usa CALL
6     RETURN   ; Retorna al bucle principal
7
8     MOVLW    d'2'
9     SUBWF    B_Estados, W
```

```

10    BTFSC    STATUS, Z
11    CALL     01a
12    RETURN
13    MOVLW    d'3'
14    SUBWF    B_Estados, W
15    BTFSC    STATUS, Z
16    CALL     Luces_Policia
17    RETURN

```

Cada secuencia se ejecuta como subrutina mediante **CALL**, permitiendo que al detectar S2 o S3, la secuencia retorne inmediatamente al selector de estados sin necesidad de saltos complejos.

3. Bucle principal mejorado:

```

1 Repetir_Estado:
2     CALL     Opcion_sec      ; Ejecuta la secuencia actual
3
4     BTFSS    PORTC, 5        ; Verifica S3
5     GOTO     Apagar_Todo
6     BTFSS    PORTC, 4        ; Verifica S2
7     GOTO     Boton_secuencias
8     GOTO     Repetir_Estado  ; Si ninguno, repite

```

Este bucle centraliza el control. Después de cada ejecución de secuencia, verifica si se presionó S3 (apagar) o S2 (cambiar secuencia). Si ninguno fue presionado, repite la secuencia actual.

4. Detección temprana en las secuencias:

```

1 Cortina:
2     BTFSS    PORTC, 5
3     RETURN   ; Retorna inmediatamente si S3
4     BTFSS    PORTC, 4
5     RETURN   ; Retorna inmediatamente si S2

```

Explicación: Cada secuencia verifica continuamente S2 y S3. Si detecta alguno, ejecuta **RETURN** inmediatamente, regresando al bucle principal para procesar el cambio de estado.

5. Reset completo al presionar S3:

```

1 Apagar_Todo:
2     CLRF     LATD            ; Apaga todos los LEDs
3     CLRF     B_Estados      ; Reinicia a estado 0
4 Boton_S3:
5     BTFSS    PORTC, 5
6     GOTO     Boton_S3
7     CALL     Delay100ms
8     GOTO     EncuestaON     ; Regresa a espera inicial

```

Explicación: Al presionar S3, además de apagar los LEDs, se reinicia B_Estados a 0 y el programa regresa a **EncuestaON**, obligando al usuario a presionar S2 nuevamente para seleccionar una secuencia desde el inicio.

3.1.4. Ventajas de la Implementación con Máquina de Estados

- **Modularidad:** Cada secuencia es independiente y puede modificarse sin afectar las demás.
- **Mantenibilidad:** Es más fácil agregar nuevas secuencias solo agregando un nuevo estado y su respectiva subrutina.
- **Control centralizado:** El flujo del programa está claramente definido en el bucle principal.
- **Respuesta rápida:** Las secuencias pueden interrumpirse inmediatamente al detectar un cambio de botón.

4. Configuración de Bits del Microcontrolador

La configuración correcta de los *Configuration Bits* es fundamental para el funcionamiento del PIC18F4550. Una configuración incorrecta puede impedir la programación o causar comportamientos inesperados del sistema.

4.0.1. Configuración de Bits

1. En el menú superior, seleccione: **Configure → Configuration Bits...**
2. Se abrirá la ventana de configuración mostrada en la Figura 9

300000	24	PLLDIV	PLL Prescaler Selection Divide by 5 (20 MHz oscillator input)	
		CPUDIV	System Clock Postscaler [Primary Oscillator Src: /1][96 MHz PLL Src: /2]	
		USBDIV	USB Clock Selection bit USB clock source comes from the 96 MHz PLL divided by 2	
300001	0F	FOSC	Oscillator Selection bit HS oscillator, PLL enabled (HSPLL)	
		FCMEN	Fail-Safe Clock Monitor Fail-Safe Clock Monitor disabled	
		IESO	Internal/External Oscil Oscillator Switchover mode disabled	
300002	1F	PWRT	Power-up Timer Enable b PWRT disabled	
		BOR	Brown-out Reset Enable Brown-out Reset enabled in hardware only (SBOREN is disabled)	
		BORV	Brown-out Reset Voltage Minimum setting	
		VREGEN	USB Voltage Regulator E USB voltage regulator disabled	
300003	1E	WDT	Watchdog Timer Enable b WDT disabled (control is placed on the SWDTEN bit)	
		WDTPS	Watchdog Timer Postscal 1:32768	
300005	81	CCP2MX	CCP2 MUX bit CCP2 input/output is multiplexed with RC1	
		PBADEN	PORTB A/D Enable bit PORTB<4:0> pins are configured as digital I/O on Reset	
		LPT1OSC	Low-Power Timer 1 Oscil Timer1 configured for higher power operation	
		MCLRE	MCLR Pin Enable bit MCLR pin enabled; RE3 input pin disabled	
300006	81	STVREN	Stack Full/Underflow Re Stack full/underflow will cause Reset	
		LVP	Single-Supply ICSP Enab Single-Supply ICSP disabled	
		ICPRT	Dedicated In-Circuit De ICPORT disabled	
		XINST	Extended Instruction Se Instruction set extension and Indexed Addressing mode disabled (Legacy mode)	
300008	0F	CP0	Code Protection bit Block 0 (000800-001FFFh) is not code-protected	
		CP1	Code Protection bit Block 1 (002000-003FFFh) is not code-protected	
		CP2	Code Protection bit Block 2 (004000-005FFFh) is not code-protected	
		CP3	Code Protection bit Block 3 (006000-007FFFh) is not code-protected	
300009	C0	CPB	Boot Block Code Protect Boot block (000000-0007FFFh) is not code-protected	
		CPD	Data EEPROM Code Protec Data EEPROM is not code-protected	
30000A	0F	WRT0	Write Protection bit Block 0 (000800-001FFFh) is not write-protected	
		WRT1	Write Protection bit Block 1 (002000-003FFFh) is not write-protected	
		WRT2	Write Protection bit Block 2 (004000-005FFFh) is not write-protected	

Figura 9: Ventana de configuración de bits en MPLAB IDE v8.92

4.0.2. Configuraciones Críticas Implementadas

Las configuraciones más importantes realizadas para esta práctica son:

- **Oscilador (FOSC):** Configurado en modo **HS** (High Speed) para utilizar el cristal externo de 20 MHz.
- **Watchdog Timer (WDT):** Deshabilitado. Si se deja habilitado, el microcontrolador se reiniciará periódicamente, interrumpiendo las secuencias.
- **Protección de Código (CP0, CP1, CP2, CP3):** Todos configurados como **is not code-protected**. Esta configuración es **CRÍTICA**. Si se activa la protección de código, el PIC18F4550 quedará bloqueado y no se podrá reprogramar ni leer su memoria.
- **Protección de Escritura (WRT0, WRT1, WRT2):** Configurados como **is not write-protected** para permitir modificaciones en la memoria Flash.
- **Brown-out Reset (BOR):** Puede configurarse según la aplicación. En esta práctica se mantuvo deshabilitado.
- **Stack Overflow/Underflow Reset:** Habilitado para detectar errores en el uso de la pila.

ADVERTENCIA CRÍTICA

Nunca activar la protección de código (*code-protect*) durante el desarrollo.

Si se configura cualquiera de los bits CP0, CP1, CP2, CP3 como *code-protected*, el microcontrolador quedará permanentemente bloqueado y no podrá ser reprogramado. La única forma de desbloquearlo sería mediante un borrador de memoria especializado, proceso que puede dañar el dispositivo. Siempre verifique que todas las opciones de protección estén en *is not code-protected* y *is not write-protected* antes de programar.

5. Pruebas y Verificaciones

Se realizaron pruebas del sistema implementado para verificar el correcto funcionamiento de todas las secuencias y el manejo de entradas digitales.

5.1. Verificación de la Secuencia de Inicio

El LED conectado a RC0 parpadea 20 veces con intervalos de 100 ms (50 ms ON, 50 ms OFF). Al finalizar, el LED permanece encendido, indicando que el sistema está listo.

5.2. Prueba de Secuencias

Secuencia SQ1 (Cortina): Los LEDs se encienden desde los extremos hacia el centro y regresan, creando un efecto de cortina. El patrón observado fue exactamente el especificado: 10000001 → 01000010 → 00100100 → 00011000 y regreso.

Secuencia SQ2 (Ola): Un LED individual se desplaza de derecha a izquierda y viceversa continuamente, simulando una ola de luz que recorre todo el Puerto D.

Secuencia SQ3 (Luces Policía): Los 4 LEDs inferiores parpadean 3 veces (simulando luces rojas), seguidos de 3 parpadeos de los 4 LEDs superiores (simulando luces azules), alternando.

5.3. Prueba de Pulsantes

Pulsante S2: Al presionarlo durante la ejecución de cualquier secuencia, el sistema cambia inmediatamente a la siguiente secuencia en orden cíclico (SQ1 → SQ2 → SQ3 → SQ1...).

Pulsante S3: Al presionarlo, todos los LEDs se apagan inmediatamente y el sistema regresa al estado inicial, esperando que se presione S2 para iniciar nuevamente desde la secuencia SQ1.

6. Códigos Completos y Videos de Funcionamiento

Los códigos completos implementados en lenguaje ensamblador para las tres partes de la práctica, así como los videos que demuestran su funcionamiento, están disponibles en el repositorio de GitHub:

Códigos fuente (.asm):

- [Repositorio completo con los 3 códigos \(Parte 1, Parte 2 y Parte 3\)](#)

Videos de demostración (.mp4):

- [Video funcionamiento Parte 1 - Secuencia Cortina](#)
- [Video funcionamiento Parte 2 y 3 - Múltiples Secuencias](#)

Repositorio principal:

- [Repositorio Principal - Todas las Prácticas](#)

7. Preguntas.

Conteste a las siguientes preguntas:

1. **¿Cuál es la función del cristal de cuarzo (XTAL) conectado al microcontrolador?**

El cristal de cuarzo proporciona una señal de reloj externa estable y precisa al microcontrolador. Funciona como un elemento resonante que oscila a una frecuencia específica (en nuestro caso 20 MHz). Esta señal de reloj es fundamental porque sincroniza todas las operaciones internas del CPU, garantizando que las instrucciones se ejecuten a una velocidad conocida y constante, lo cual es crítico para las subrutinas de demora y para mantener la precisión temporal del sistema.

2. **¿Se podría prescindir del cristal de cuarzo para operar este microcontrolador?**

Sí, el PIC18F4550 puede operar sin cristal de cuarzo externo utilizando su oscilador interno de 8 MHz. Sin embargo, esta opción presenta desventajas: el oscilador interno es menos preciso y menor velocidad máxima de operación. Para aplicaciones que requieren alta precisión temporal o comunicación USB, es indispensable usar un cristal externo, ya que proporciona mayor estabilidad y exactitud en la frecuencia de operación.

3. **Explique cuál es y cómo se obtiene la velocidad del oscilador del CPU.**

La velocidad del CPU no es igual a la frecuencia del cristal. El PIC18F4550 divide internamente la frecuencia del oscilador por 4 para generar el ciclo de instrucción. La fórmula es:

$$F_{cy} = \frac{F_{osc}}{4} = \frac{20 \text{ MHz}}{4} = 5 \text{ MHz}$$

Esto significa que con un cristal de 20 MHz, cada instrucción se ejecuta en:

$$T_{cy} = \frac{1}{5 \text{ MHz}} = 0,2\mu s = 200ns$$

4. **¿Se podría colocar directamente un cristal de 48 MHz?**

No, no se puede conectar directamente un cristal de 48 MHz al PIC18F4550. Según la hoja de datos, el módulo oscilador en modo HS (High Speed) soporta cristales de hasta 20 MHz como máximo (Ver Figura 10).

Osc Type	Crystal Freq	Typical Capacitor Values Tested:	
		C1	C2
XT	4 MHz	27 pF	27 pF
HS	4 MHz	27 pF	27 pF
	8 MHz	22 pF	22 pF
	20 MHz	15 pF	15 pF
Capacitor values are for design guidance only. These capacitors were tested with the crystals listed below for basic start-up and operation. These values are not optimized. Different capacitor values may be required to produce acceptable oscillator operation. The user should test the performance of the oscillator over the expected V _{DD} and temperature range for the application. See the notes following this table for additional information.			
Crystals Used:			
4 MHz			
8 MHz			
20 MHz			

Figura 10: CAPACITOR SELECTION FOR CRYSTAL OSCILLATOR

Para obtener 48 MHz (necesario para el módulo USB), se debe usar un cristal de frecuencia menor (típicamente 4, 8, 12 o 20 MHz) y configurar el PLL (Phase-Locked Loop) interno que es la etapa divisora (Prescaler) del microcontrolador para multiplicar la frecuencia del PLL de 96 MHz interno. El PLL toma la señal del cristal y la multiplica internamente para generar los 48 MHz requeridos por el módulo USB. La hoja de datos proporciona las siguientes configuraciones permitidas para alimentar el PLL (el cual requiere estrictamente 4 MHz a su entrada):

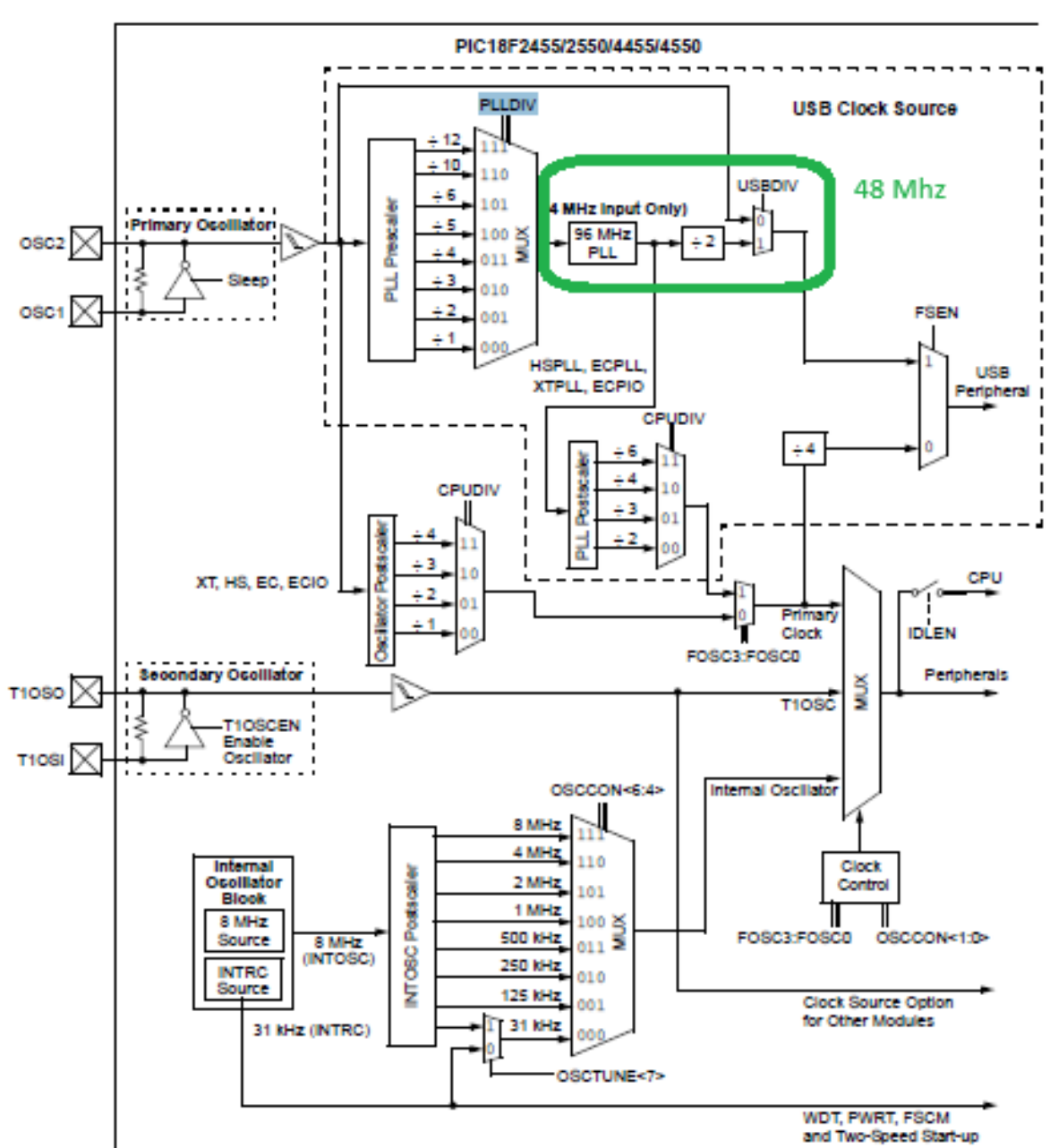


Figura 11: Diagrama de bloques del sistema de reloj y módulo PLL del PIC18F4550.

TABLE 2-3: OSCILLATOR CONFIGURATION OPTIONS FOR USB OPERATION

Input Oscillator Frequency	PLL Division (PLLDIV2:PLLDIV0)	Clock Mode (FOSC3:FOSC0)	MCU Clock Division (CPUDIV1:CPUDIV0)	Microcontroller Clock Frequency
48 MHz	N/A ⁽¹⁾	EC, ECIO	None (00)	48 MHz
			+2 (01)	24 MHz
			+3 (10)	16 MHz
			+4 (11)	12 MHz
48 MHz	+12 (111)	EC, ECIO	None (00)	48 MHz
			+2 (01)	24 MHz
			+3 (10)	16 MHz
			+4 (11)	12 MHz
		ECPLL, ECPIO	+2 (00)	48 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
40 MHz	+10 (110)	EC, ECIO	+5 (11)	16 MHz
			None (00)	40 MHz
			+2 (01)	20 MHz
			+3 (10)	13.33 MHz
			+4 (11)	10 MHz
		ECPLL, ECPIO	+2 (00)	40 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
24 MHz	+6 (101)	HS, EC, ECIO	+5 (11)	16 MHz
			None (00)	24 MHz
			+2 (01)	12 MHz
			+3 (10)	8 MHz
			+4 (11)	6 MHz
		HSPLL, ECPLL, ECPIO	+2 (00)	48 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
			+5 (11)	16 MHz

(a) Selección del divisor para el Prescaler del PLL.

TABLE 2-3: OSCILLATOR CONFIGURATION OPTIONS FOR USB OPERATION (CONTINUED)

Input Oscillator Frequency	PLL Division (PLLDIV2:PLLDIV0)	Clock Mode (FOSC3:FOSC0)	MCU Clock Division (CPUDIV1:CPUDIV0)	Microcontroller Clock Frequency
20 MHz	+5 (100)	HS, EC, ECIO	None (00)	20 MHz
			+2 (01)	10 MHz
			+3 (10)	6.67 MHz
			+4 (11)	5 MHz
		HSPLL, ECPLL, ECPIO	+2 (00)	48 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
			+5 (11)	16 MHz
16 MHz	+4 (011)	HS, EC, ECIO	None (00)	16 MHz
			+2 (01)	8 MHz
			+3 (10)	5.33 MHz
			+4 (11)	4 MHz
		HSPLL, ECPLL, ECPIO	+2 (00)	48 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
			+5 (11)	16 MHz
12 MHz	+3 (010)	HS, EC, ECIO	None (00)	12 MHz
			+2 (01)	6 MHz
			+3 (10)	4 MHz
			+4 (11)	3 MHz
		HSPLL, ECPLL, ECPIO	+2 (00)	48 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
			+5 (11)	16 MHz
8 MHz	+2 (001)	HS, EC, ECIO	None (00)	8 MHz
			+2 (01)	4 MHz
			+3 (10)	2.67 MHz
			+4 (11)	2 MHz
		HSPLL, ECPLL, ECPIO	+2 (00)	48 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
			+5 (11)	16 MHz
4 MHz	+1 (000)	XT, HS, EC, ECIO	None (00)	4 MHz
			+2 (01)	2 MHz
			+3 (10)	1.33 MHz
			+4 (11)	1 MHz
		HSPLL, ECPLL, XTPLL, ECPIO	+2 (00)	48 MHz
			+3 (01)	32 MHz
			+4 (10)	24 MHz
			+5 (11)	16 MHz

(b) CONFIGURATION OPTIONS FOR USB OPERATION.

Figura 12: Configuraciones PLLDIV y selección de frecuencias según la hoja de datos.

5. ¿Cuál es la velocidad máxima que se lograría utilizando el oscilador interno de 8 MHz?

Utilizando el bloque del oscilador interno (INTOSC), la frecuencia máxima que se puede configurar es de 8 MHz. Con esta frecuencia, la velocidad del CPU (ciclo de instrucción) sería:

$$F_{cy} = \frac{8 \text{ MHz}}{4} = 2 \text{ MHz}$$

Por lo tanto, cada instrucción se ejecutaría aproximadamente cada:

$$T_{cy} = \frac{1}{2 \text{ MHz}} = 0,5\mu s$$

6. ¿Por qué es necesario configurar los registros TRIS para manipular los puertos de entrada y salida digitales del microcontrolador? Por ejemplo TRISB.

Los registros TRIS (TRI-State) determinan la dirección del flujo de datos en cada pin del microcontrolador. Cada bit del registro TRIS controla un pin correspondiente del puerto:

- TRIS = 1: El pin se configura como entrada (permite leer señales externas)
- TRIS = 0: El pin se configura como salida (puede escribir niveles lógicos 0 o 1)

Esta configuración es necesaria porque el hardware interno del microcontrolador debe conectar o desconectar los buffers de entrada/salida según la función deseada. Sin esta configuración, el pin quedaría en un estado indefinido. En nuestra práctica, CLRF TRISD configuró todo el Puerto D como salidas para controlar los LEDs.

7. ¿Por qué los puertos de entrada usados en la práctica (RC4 y RC5) no tienen un registro TRIS asociado?

Los pines RC4 y RC5 son especiales en el PIC18F4550 porque forman parte del módulo USB. Para usarlos como entradas digitales generales, es necesario desactivar el módulo USB mediante los registros:

- BCF UCON, 3: Desactiva el módulo USB
- BSF UCFG, 3: Configura los pines para uso digital general

Una vez desactivado el USB, estos pines se configuran automáticamente como entradas digitales sin necesidad de modificar TRISC. Esta es una característica específica de estos pines debido a su función dual USB/Digital.

8. **Explique cómo consiguió que un mismo micropulsante (entrada digital) pueda utilizarse para diferentes funciones de su programa (en este caso, las secuencias de salida).**

Se implementó un sistema de máquina de estados usando la variable B_Estados. El pulsante S2 incrementa esta variable cada vez que se presiona, y el programa evalúa el valor actual para decidir qué secuencia ejecutar:

- Cuando S2 se presiona, se ejecuta INCF B_Estados, 1
- Si B_Estados = 1, ejecuta la secuencia Cortina
- Si B_Estados = 2, ejecuta la secuencia Ola
- Si B_Estados = 3, ejecuta la secuencia Luces Policía
- Si B_Estados llega a 4, se reinicia a 1 (ciclo infinito)

Este método permite que el mismo pulsante tenga múltiples funciones contextuales dependiendo del estado actual del sistema. La clave está en mantener una variable de estado persistente que recuerde en qué secuencia se encuentra. Adicionalmente a estos botones se deben poner un delay debido a que al tener unas placas estas mandan muchas señales (0 - 1) hasta soltar el boton por ende entra en estado **transitorio (Antirrebote)** y con el delay esperamos que pase ese estado transitorio del pulsante.

9. **Explique las diferencias que encontró en el lenguaje ensamblador, sobre los otros lenguajes de programación convencionales que conoce; concretamente, sobre la forma de implementar estructuras de control: if, then, while, etc.**

El lenguaje ensamblador del PIC18F no tiene estructuras de control de alto nivel como if, while o for. Todas estas estructuras deben implementarse manualmente usando:

Instrucciones de salto condicional:

- BTFSS (Bit Test File Skip if Set): Equivalente a if (bit == 1)
- BTFSC (Bit Test File Skip if Clear): Equivalente a if (bit == 0)
- DECFSZ (Decrement File Skip if Zero): Equivalente a un while con contador -1

Ejemplo comparativo:

En C (alto nivel):

```
1 if (PORTC.4 == 0) {  
2     ejecutarSecuencia();  
3 }
```

En Ensamblador:

```
1 BTFSC    PORTC, 4      ; Salta si bit 4 esta en 1  
2 GOTO     ejecutarSecuencia ; Continúa si bit 4 esta en 0
```

Las principales diferencias son:

- a) **Control manual del flujo:** No hay estructuras predefinidas; se usan etiquetas y saltos explícitos.
- b) **Lógica invertida:** Las instrucciones BTFSS/BTFSC "saltan" si la condición es verdadera, no ejecutan un bloque.
- c) **Menor abstracción:** El programador debe pensar en términos de bits individuales y saltos de programa.
- d) **Mayor eficiencia:** Cada instrucción corresponde directamente a una operación del CPU, sin overhead del compilador.
- e) **Dificultad de lectura:** El código es más difícil de entender sin comentarios, ya que no hay nombres descriptivos para las estructuras de control.

En nuestra práctica, implementar el `while` de la secuencia Cortina requirió crear etiquetas (`Cortina:`, `Bucle.Cortina:`) y saltos condicionales (`GOTO`), algo que en lenguajes de alto nivel sería una simple estructura `while(true)`.

8. Conclusiones y Recomendaciones

8.1. Conclusiones

- Se logró implementar correctamente un sistema de entradas y salidas digitales utilizando el microcontrolador PIC18F4550 y programación en lenguaje ensamblador.
- Se comprendió el funcionamiento de los registros TRIS, LAT y PORT para la configuración y control de puertos digitales del microcontrolador.
- Se verificó el funcionamiento del oscilador externo de 20 MHz y la importancia de los capacitores de 22 pF para garantizar una señal de reloj estable.
- Se desarrollaron secuencias utilizando LEDs y pulsantes, aplicando subrutinas de demora y control con lo aprendido y mediante máquina de estados.
- Se comprobó que el lenguaje ensamblador ofrece control directo del hardware, aunque requiere una programación más detallada que los lenguajes de alto nivel.
- La implementación de una máquina de estados permitió organizar mejor el programa y facilitar el cambio entre múltiples secuencias.

8.2. Recomendaciones

- Verificar cuidadosamente las conexiones del circuito antes de energizar el sistema, especialmente las líneas de alimentación y programación del PIC.
- Configurar correctamente los *Configuration Bits*, asegurándose de utilizar el oscilador adecuado y deshabilitar protecciones innecesarias durante el desarrollo.
- Utilizar comentarios descriptivos en el código ensamblador para facilitar la comprensión y mantenimiento del programa.
- Implementar resistencias limitadoras y resistencias pull-up adecuadas para proteger los LEDs y garantizar lecturas estables en los pulsantes.

9. Referencias

Referencias

- [1] Microchip Technology Inc., *PIC18F2455/2550/4455/4550 Data Sheet*, DS39632E, 2023. [En línea]. Disponible en: <https://ww1.microchip.com/downloads/en/devicedoc/39632e.pdf>
- [2] Microchip Technology Inc., *PICkit 3 Programmer/Debugger User's Guide*, DS51795A, 2023. [En línea]. Disponible en: <https://ww1.microchip.com/downloads/en/DeviceDoc/51795B.pdf>
- [3] Microchip Technology Inc., *MPLAB IDE v8.92 User's Guide*, 2024. [En línea]. Disponible en: <https://www.microchip.com/mplab>
- [4] PIcStregone Dom, *PIC18F4550: Oscilador para USB y CPU*, 2016. [En línea]. Disponible en: <https://stregonedom.wordpress.com/2016/09/29/pic18f4550-oscilador-para-usb-y-cpu/>